Software Engineering Processes

Course Code: XB_0089

Claudia Raibulet & Fernanda Madeiral

Computer Science Department
Vrije Universiteit Amsterdam

Bachelor in Computer Science, June 5th 2024

Part 1: What is Software Quality?

What is Software Quality?

- **Software quality** refers to the **degree** to which **software conforms** to its **requirements** and meets the **needs of its users**.

What is Software Quality?

- Definition 1: “The **capability** of a software product to satisfy **stated and implied needs** when used **under specified conditions**.”

- Definition 2: Software quality depends on “the **degree** to which those established **requirements accurately represent stakeholder needs, wants, and expectations**.”

What is Software Quality?

- High quality software meets its requirements, which in turn should accurately reflect stakeholder needs.
- Quality is about aligning the software with both its formal requirements as well as true user needs.

Part 2: Errors, Defects, Failures

What Are Errors, Defects, Failures?

- **Error** – A human mistake made by a developer.
  - An example is misunderstanding a requirement and coding to the wrong specification.
- **Defect** – A flaw or imperfection inserted into a software due to an error.
  - An example is a bug in the code or issues with other artifacts like requirements. Defects get inserted when errors are made.
- **Failure** – The termination of the software's ability to function as intended.
  - Occur when the software executing encounters a defect.
  - The user experiences the software failing in some unintended way.

Part 3: Software Quality Management

Software Quality Management

- **Quality planning** – Defining quality objectives, requirements, targets, and planning of quality assurance.
- **Quality control** – Techniques to measure quality characteristics, review work products, and find defects.
- **Quality assurance** – Processes to ensure compliance with procedures.
- **Quality improvement** – Defect analysis and process enhancements.
- **Resources** – Infrastructure, tools, training that enable quality processes.
- **Standards** – Regulations, models, certifications that guide quality work.
- **Culture** – Values, behaviors that encourage quality mindset.

How to Control Software Quality?

- Testing the software
- Static analysis of software
- Dynamic analysis of software
- ...

Part 4: Software Evaluation

What is Evaluation?

- The making of a judgement about the amount, number, or value of something
- Assessment, judgement, rating, …

How is Software Evaluated?

- Various points of view:
  - Functionality
  - Performance
  - Quality
  - Usability
  - Maintainability
  - …
- Various approaches:
  - Quantitative
  - Qualitative
  - …

Part 4: Software Metrics

What Represents a Good Software Metric?

- Comparability
  - Allows comparisons between software systems
  - Metric values can be compared
- Intuitive interpretation
  - Concerns what the metric tells you about the system
  - A metric should be interpreted also by a non-expert in the domain
- Simple and efficient computation
  - Compute the metric with little effort

Examples of Software Metrics

- **LOC** – lines of code (or NLOC – number of lines of code)

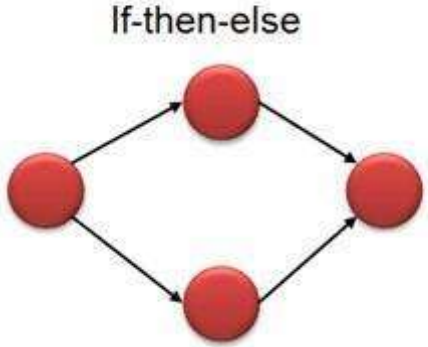
Examples of Software Metrics

- Chidamber and Kemerer (**CK**) Metrics Suite
  - Metrics for object-oriented software
  - Examples:
    - **Weighted Methods per Class (WMC)**: measures the complexity of a class by assessing the sum of complexities of its methods
    - **Number Of Children (NOC)**: represents the number of immediate subclasses a class has; provides an understanding of class reuse and potential dependencies
    - **Coupling Between Object Classes (CBO)**: measures the number of classes to which a class is coupled; helps in evaluating the level of interdependence among classes
- ...

Examples of Software Metrics

- McCabe Cyclomatic Complexity Metric
  - A quantitative measure of independent paths in the source code of the software product
  - An independent path is that path which has at least one edge that has not been yet traversed in any other path

- V(G) = E – N + 2
  - E = number of edges,
  - N = number of nodes



Tools for Software Evaluation & Quality Assessment

- Understand
- SonarQube

Understand Tool

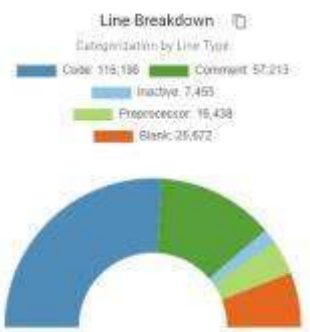
Understand™
by SciTools

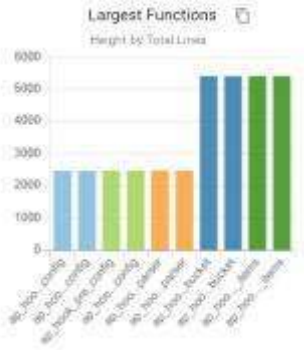
Understand code.

- Discover how **Legacy Code** fits together
- Visualize with customizable **Graphs**
- **Comply** with AUTOSAR, MISRA, and more
- Automate solutions with full **API** access
- Gain valuable insight with **Metrics**
- **Onboard** new team members quickly

START NOW

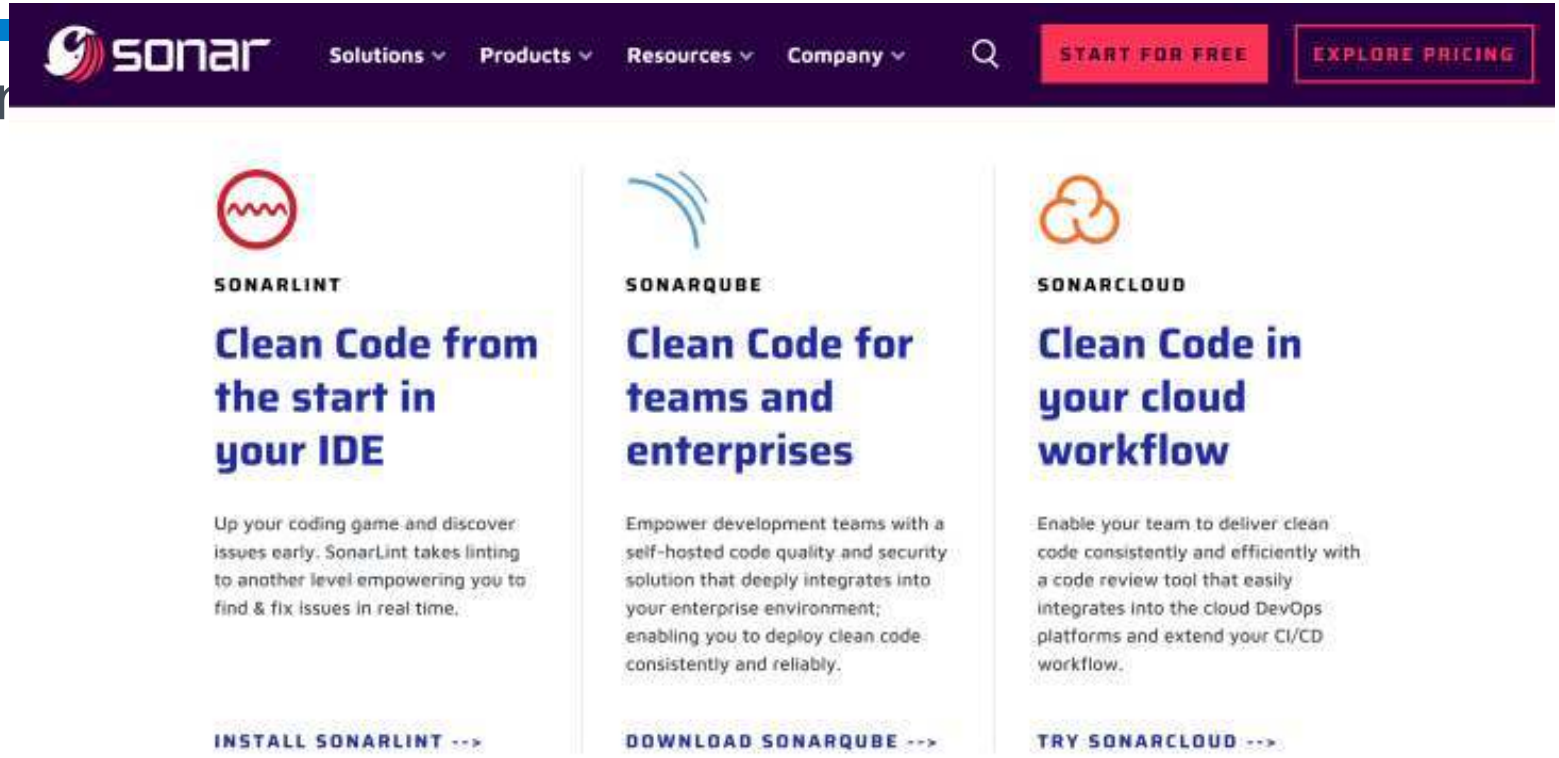
Get a demo ⟶





https://scitools.com/

SonarQube



https://www.sonarsource.com/

Part 5: Quality Attributes

Quality Attributes

- ISO/IEC 25010
- https://iso25000.com/index.php/en/iso-25000-standards/iso-25010

SOFTWARE PRODUCT QUALITY								
FUNCTIONAL SUITABILITY	PERFORMANCE EFFICIENCY	COMPATIBILITY	INTERACTION CAPABILITY	RELIABILITY	SECURITY	MAINTAINABILITY	FLEXIBILITY	SAFETY
FUNCTIONAL COMPLETENESS	TIME BEHAVIOUR	CO-EXISTENCE	APPROPRIATENESS RECOGNIZABILITY	FAULTLESSNESS	CONFIDENTIALITY	MODULARITY	ADAPTABILITY	OPERATIONAL CONSTRAINT
FUNCTIONAL CORRECTNESS	RESOURCE UTILIZATION	INTEROPERABILITY	LEARNABILITY	AVAILABILITY	INTEGRITY	REUSABILITY	SCALABILITY	RISK IDENTIFICATION
FUNCTIONAL APPROPRIATENESS	CAPACITY		OPERABILITY	FAULT TOLERANCE	NON-REPUDIATION	ANALYSABILITY	INSTALLABILITY	FAIL SAFE
			USER ERROR PROTECTION	RECOVERABILITY	ACCOUNTABILITY	MODIFIABILITY	REPLACEABILITY	HAZARD WARNING
			USER ENGAGEMENT		AUTHENTICITY	TESTABILITY		SAFE INTEGRATION
			INCLUSIVITY		RESISTANCE			
			USER ASSISTANCE					
iso25000.com			SELF-DESCRIPTIVENESS					

Performance Efficiency

- Represents the **degree** to which a software **performs its functions within specified time** and is **efficient in the use of resources** (such as CPU, memory, storage, network devices, energy, materials…) **under specified conditions**.
- **Time behaviour -** Degree to which the response time and throughput rates of a software, when performing its functions, meet requirements.
- **Resource utilization -** Degree to which the amounts and types of resources used by a software, when performing its functions, meet requirements.
- **Capacity -** Degree to which the maximum limits of a software parameter meet requirements.

Part 5: Code Smells and Refactoring

Code Smells

- Indicators of **potential issues** (deeper problems) **in the code over time**
- Results of poor or misguided programming (e.g., not conforming to the SOLID principles)
  1. **S**ingle Responsibility
  2. **O**pen/Closed
  3. **L**iskov Substitution
  4. **I**nterface Segregation
  5. **D**ependency Inversion

Examples of Code Smells

- Large/God Class
- Long Method
- Duplicated Code
- Switch Statement
- Feature Envy
- ...

Example 1

```
public class Student {
        private String name, surname;
        private String streetName;
        private int streetNumber;
        private int grades[];

        public void Student () {...}
        public boolean addGrade (int grade) {...}
        public boolean removeGrade (int grade) {...}
        public void listGrade () {...}
        public boolean changeAddress (String newName, int newNumber) {...}
        ...
}
```

Large Class Code Smell

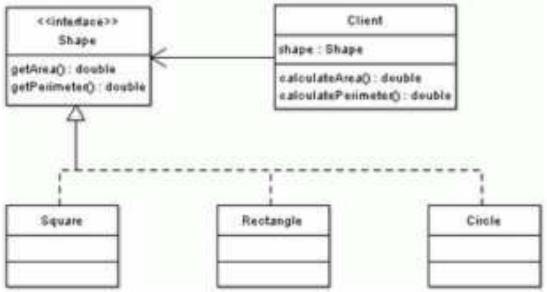
Example 2

```
public class Client {
 private double a;
 private double b;
 private double r;
 ...
 public double calculateArea(int shape) {
     double area = 0;
     switch(shape) {
         case SQUARE:
             area = a * a;
             break;
         case RECTANGLE:
             area = a * b;
             break;
         case CIRCLE:
             area = Math.PI * r * r;
             break;
     }
     return area;
 }

 public double calculatePerimeter(int shape) {
     double perimeter = 0;
     switch(shape) {
         case SQUARE:
             perimeter = 4 * a;
             break;
         case RECTANGLE:
             perimeter = 2 * (a + b);
             break;
         case CIRCLE:
             perimeter = 2 * Math.PI * r;
             break;
     }
     return perimeter;
 }
 ...
}
```

Switch Statement Code Smell

Example 2 Solution



Shape.java file

```
public interface Shape {
 public double getArea();
 public double getPerimeter();
}
```

Square.java file

```
public class Square implements Shape {
 private double a;
 ...
 public double getArea() {
     return a * a;
 }
 public double getPerimeter() {
     return 4 * a;
 }
}
```

Rectangle.java file

```
public class Rectangle implements Shape {
 private double a;
 private double b;
 ...
 public double getArea() {
     return a * b;
 }
 public double getPerimeter() {
     return 2 * (a + b);
 }
}
```

```
public class Client {
 private double a;
 private double b;
 private double r;
 ...
 public double calculateArea(int shape) {
     double area = 0;
     switch(shape) {
         case SQUARE:
             area = a * a;
             break;
         case RECTANGLE:
             area = a * b;
             break;
         case CIRCLE:
             area = Math.PI * r * r;
             break;
     }
     return area;
 }

 public double calculatePerimeter(int shape) {
     double perimeter = 0;
     switch(shape) {
         case SQUARE:
             perimeter = 4 * a;
             break;
         case RECTANGLE:
             perimeter = 2 * (a + b);
             break;
         case CIRCLE:
             perimeter = 2 * Math.PI * r;
             break;
     }
     return perimeter;
 }
 ...
}
```

Example 3

Feature Envy Code Smell

```
public class Employee {
        private AddressofficeAddress = null;
        [...]
        public String getMailingAddress() {
           StringBuilder mailingAddress = new StringBuilder();
           mailingAddress.append(officeAddress.BuildingName + " - "  + officeAddress.Office);
           mailingAddress.append("\n");
           mailingAddress.append(officeAddress.BuildingNumber + " " + officeAddress.StreetName);
           mailingAddress.append("\n");
           mailingAddress.append(officeAddress.City + ", " + officeAddress.State);
           mailingAddress.append("\n");
           mailingAddress.append(officeAddress.PostalCode);
           return mailingAddress.toString();
        }
        [...]
}
```

Example 3 - Solution

```
public class Address {
        [...]
        public String getMailingAddress() {
                StringBuilder address = new StringBuilder();
                address.append(BuildingName + " - "  + Office);
                address.append("\n");
                address.append(BuildingNumber + " " + StreetName);
                address.append("\n");
                address.append(City + ", " + State);
                address.append("\n");
                address.append(PostalCode);
                return address.toString();
        }
        [...]
}
```

```
public class Employee {
        private Address officeAddress = null;
        [...]
        public String getMailingAddress() {
           return  officeAddress.getMailingAddress();
        }
        [...]
}
```

Example 4

```
int[] array1 = {0,1,2,3};
int[] array2 = {4,5,2,35};

int sum1 = 0;
int sum2 = 0;
int average1 = 0;
int average2 = 0;

for (int i = 0; i < array1.length ; i++)
{
    sum1 += array1[i];
}
average1 = sum1/array2.length;

for (int i = 0; i < array2.length; i++)
{
    sum2 += array2[i];
}
average2 = sum2/array2.length;
```

Duplicated Code - Code Smell

Example 4 - Solution

```
int[] array1 = {0,1,2,3};
int[] array2 = {4,5,2,35};

int average1 = calcAverage(array1);
int avarage2 = calcAverage(array2);


...

pubic int calcAverage(int[] array) {
    int sum = 0;
    for (int i = 0; i < array.length; i++)
    {
        sum += array[i];
    }
    return sum/array.length;
}
```

Refactoring

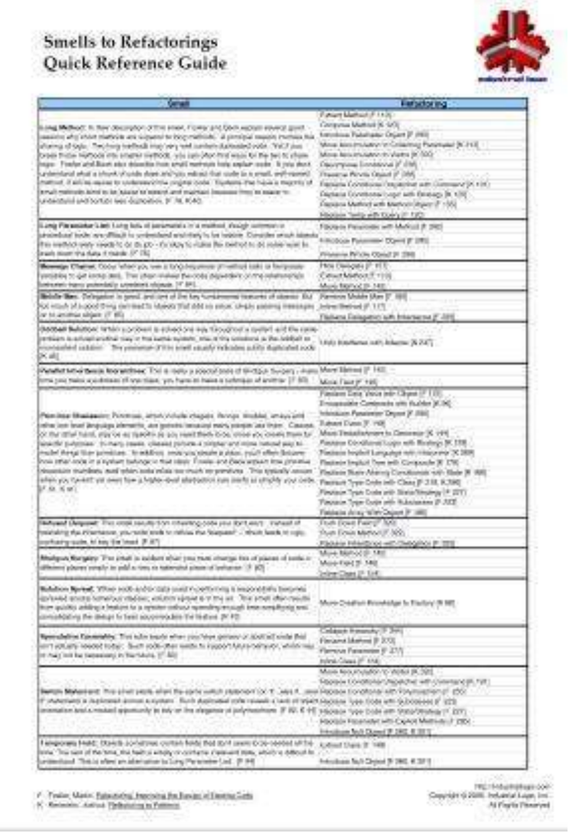
- Performs structural changes to improve the quality of the code
- The external behavior remains unchanged

- Have a look at:
  - https://refactoring.guru/refactoring/what-is-refactoring

Code Smells and Refactoring


Smells to Refactorings
Quick Reference Guide

Smell	Refactoring


Smells to Refactorings
Quick Reference Guide

Smell	Refactoring

https://www.industriallogic.com/img/blog/2005/09/smellstorefactorings.pdf

To Do

Reading – For the 3rd Lecture

- Exam material:
  - Shyam R. Chidamber, Chris F. Kemerer, A METRICS SUITE FOR OBJECT ORIENTED DESIGN
- Additional reading (highly recommended):
  - All the links indicated in the presentation

Takeaways?